Ashwath Raj M

AP25110010712 | Core CSE A

LAB 12

Implementation of N-Queen Problem using Backtracking

Course	CSE-204 - Design and Analysis of Algorithms
Student	Ashwath Raj M | AP25110010712
Section	Core CSE A

Problem

Implement the N-Queen problem using backtracking. Place N queens on an N x N chessboard so that no two queens attack each other. Display all valid arrangements and the total number of solutions.

Method

The program places one queen in each row. For every row, it checks each column. A position is safe when no previously placed queen is in the same column or on either diagonal. If a safe position is found, the program places the queen and recursively processes the next row. If no valid arrangement can be completed, it removes the last queen and tries another position.

Output:

```
Enter number of queens:
Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

Total solutions: 2
```

Output Screenshot

GitHub Repository

https://github.com/Ashwath-Raj/CSE-204-DAA-assignments-

Code:

```
/*
Program Name: N-Queen Problem using Backtracking
Program Description: Places N queens on an N x N chessboard so that
                     no two queens attack each other.
Program Creation Time: Nov 13, 2026
Program Creator: Ashwath Raj | AP25110010712
*/

#include <iostream>
#include <vector>
#include <cstdlib>
using namespace std;

void printBoard(const vector<int>& board, int n, int solution) {
    cout << "\nSolution " << solution << ":\n";
    for (int row = 0; row < n; ++row) {
        for (int col = 0; col < n; ++col)
            cout << (board[row] == col ? "Q " : ". ");
        cout << '\n';
    }
}

bool isSafe(const vector<int>& board, int row, int col) {
    for (int r = 0; r < row; ++r) {
        int c = board[r];
        if (c == col || abs(c - col) == abs(r - row))
            return false;
    }
    return true;
}

void solveNQueens(vector<int>& board, int row, int n, int& count) {
    if (row == n) {
        printBoard(board, n, ++count);
        return;
    }

    for (int col = 0; col < n; ++col) {
        if (isSafe(board, row, col)) {
            board[row] = col;
            solveNQueens(board, row + 1, n, count);
            board[row] = -1;
        }
    }
}

int main() {
    int n;
    cout << "Enter number of queens: ";
    cin >> n;

    if (!cin || n <= 0 || n > 15) {
        cerr << "Enter a valid N between 1 and 15.\n";
        return 1;
    }

    vector<int> board(n, -1);
    int solutionCount = 0;
    solveNQueens(board, 0, n, solutionCount);

    if (solutionCount == 0)
        cout << "\nNo solution exists for N = " << n << ".\n";

    cout << "\nTotal solutions: " << solutionCount << '\n';
    return 0;
}
```

Complexity

Time Complexity: O(N!) upper-bound estimate for the backtracking search. The algorithm places one queen per row and prunes unsafe positions.
Space Complexity: O(N) auxiliary space for the queen-position array and recursion stack, excluding printed output.